Part 1: Introduction to Python Programming

Introduction

Python is a high-level programming language widely used for data processing, scientific computing, automation, and data analysis.

Python is popular because its syntax is simple and easy to read. It also provides many libraries for working with different types of data.

In this course, we use Python to collect, organize, process, and explore datasets.

Why Python for Data Processing?

Python allows us to:

- Store and manipulate data.
- Perform mathematical calculations.
- Work with lists and dictionaries.
- Automate repetitive tasks.
- Read and process different data formats.
- Explore datasets using Pandas.
- Create visualizations using Matplotlib.

Learning Objectives

After completing this practical session, you should be able to:

1. Execute basic Python commands in JupyterLab.
2. Create variables and identify their data types.
3. Perform mathematical operations.
4. Convert values between data types.
5. Work with strings, lists, and dictionaries.
6. Use conditional statements and loops.
7. Create and call simple Python functions.
8. Apply basic Python operations to small datasets.

1. Your First Python Program

A Python program consists of instructions that are executed by the Python interpreter.

In JupyterLab, we write Python code inside **code cells**.

To execute a code cell, press **Shift + Enter**.

The `print()` Function

The `print()` function displays text, numbers, or calculation results.

Text values must be enclosed in quotation marks, either single quotes (`' '`) or double quotes (`" "`).

Python executes instructions from top to bottom within a code cell.

In [7]:
```
print("Hello, Python!")
print("Welcome to Data Processing with Python")
print("University of Eastern Finland")

print(10)
print(10 + 20)
```

```
Hello, Python!
Welcome to Data Processing with Python
University of Eastern Finland
10
30
```

2. Comments in Python

Comments are notes written inside a program to explain the code.

Python ignores comments during execution.

A single-line comment starts with the `#` symbol.

Comments are useful for:

- Explaining what a program does.
- Describing calculations.
- Making code easier to understand.
- Helping other people read and maintain the program.

Use short and meaningful comments rather than explaining every individual line.

3. Variables in Python

A variable is a name used to store a value in a program.

For example, we can use variables to store a student's name, age, or examination score.

Creating Variables

Python uses the assignment operator `=` to assign a value to a variable.

```
name = "Anna"
age = 22
```

In Python, we do not need to declare the data type of a variable before assigning a value.

Rules for Variable Names

- A variable name can contain letters, numbers, and underscores.
- A variable name cannot start with a number.
- Variable names cannot contain spaces.
- Python variable names are case-sensitive.
- Python keywords such as `if`, `for`, and `class` cannot be used as variable names.

Recommended Naming Style

Use meaningful variable names with underscores.

Examples:

- `student_name`
- `patient_age`
- `sample_temperature`
- `total_score`

In [8]:
```
student_name = "Anna"
student_age = 22
student_score = 85

print("Student:", student_name)
print("Age:", student_age)
print("Score:", student_score)
```

```
Student: Anna
Age: 22
Score: 85
```

In [9]:
```
score = 70

print("Original score:", score)

score = 85

print("Updated score:", score)

score = score + 5

print("Final score:", score)
```

```
Original score: 70
Updated score: 85
Final score: 90
```

4. Basic Data Types

Every value in Python has a data type.

The data type determines what kind of information a variable stores and which operations can be performed on it.

Common Python Data Types

Data Type	Description	Example
`int`	Integer or whole number	`25`
`float`	Decimal number	`36.5`
`str`	Text or string	`"Anna"`
`bool`	Boolean value	`True` or `False`
`list`	Ordered collection	`[10, 20, 30]`
`dict`	Key-value collection	`{"name": "Anna"}`
`NoneType`	Represents no value	`None`

The `type()` Function

The `type()` function identifies the data type of a value.

Understanding data types is important because data-processing operations often require particular types.

For example, we cannot directly add a string containing `"20"` to the integer `10`.